



مقدمه

در این پروژه قصد داریم با نحوه‌ی مدیریت پردازش‌ها (Processes) و روش‌های ارتباط بین پردازش‌های (Inter-Process Communication) در سیستم‌عامل‌های شبه یونیکس آشنا شویم. در بسیاری از سیستم‌های واقعی، داده‌ها به صورت جریانی (Streaming) تولید می‌شوند و پردازش آن‌ها باید به صورت مرحله‌ای و همزمان انجام شود. در چنین سیستم‌هایی، داده‌ها به صورت تدریجی بین چندین پردازش منتقل شده و هر پردازش بخشی از پردازش را انجام می‌دهد.

در این تمرین، شما سیستمی چندمرحله‌ای طراحی خواهید کرد که داده‌های مربوط به کتاب‌ها و امتیاز کاربران را پردازش کرده و یک معیار نهایی برای هر کتاب محاسبه می‌کند. این پردازش باید به صورت on-the-fly انجام شود؛ به این معنا که داده‌ها بلافاصله پس از دریافت پردازش شده و نیازی به انتظار برای خواندن کامل فایل‌ها وجود ندارد.

برای پیاده‌سازی این سیستم باید از فراخوانی‌های سیستمی مانند `fork`، `exec` و همچنین `pipe` ها برای ارتباط بین پردازش‌ها استفاده کنید. هدف این پروژه پیاده‌سازی یک pipeline واقعی مبتنی بر `pipe` و `process` است، نه صرفاً تولید خروجی نهایی.

هدف پروژه

هدف این تمرین آشنایی عملی با مدیریت پردازش‌ها (Process Management) و ارتباط بین پردازش‌های (IPC) در سیستم‌عامل‌های شبه یونیکس است. در این پروژه دانشجو با استفاده از فراخوانی‌های سیستمی `fork`، `exec` و `pipe` یک pipeline چندمرحله‌ای طراحی می‌کند که داده‌ها را به صورت جریانی (on-the-fly) پردازش کرده و بین چند پردازش مستقل منتقل می‌کند.

نتایج مورد انتظار

در پایان اجرای برنامه، برای هر book_id موجود در داده‌ها، میانگین قیمت و میانگین امتیاز کاربران توسط پردازش‌های مختلف محاسبه شده و در پردازش Stage 2 ترکیب می‌شوند. سپس امتیاز نهایی هر کتاب بر اساس رابطه

$$\text{BookScore} = \text{MeanPrice} / \text{MeanRating}$$

محاسبه شده و نتیجه نهایی برای هر کتاب توسط پردازش Father در خروجی ترمینال نمایش داده می‌شود.

معماری پردازش مرحله‌ای

در این پروژه پردازش داده‌ها در قالب یک pipeline چهار مرحله‌ای انجام می‌شود. این سیستم شامل یک پردازش اصلی و سه پردازش فرزند است:

- پردازش اصلی (Father)
- پردازش Stage 1
- پردازش Stage 2
- پردازش Stage 3

هر یک از این پردازش‌ها وظیفه‌ی مشخصی در پردازش داده‌ها دارند و از طریق pipe با یکدیگر ارتباط برقرار می‌کنند. هر Stage باید به صورت یک پردازش مستقل اجرا شود و تنها از طریق pipe با سایر مراحل ارتباط داشته باشد.

شرح پروژه

فرض کنید یک سامانه‌ی تحلیل داده برای کتاب‌ها در اختیار داریم که اطلاعات دو منبع مختلف را پردازش می‌کند:

1. اطلاعات قیمت کتاب‌ها
2. اطلاعات امتیاز کاربران به کتاب‌ها

این اطلاعات در قالب دو فایل CSV در اختیار برنامه قرار می‌گیرد. فایل اول شامل اطلاعات قیمت کتاب‌ها است. در این فایل ممکن است برای هر کتاب چندین رکورد وجود داشته باشد (برای مثال قیمت آن کتاب در فروشگاه‌های مختلف).
فرمت این فایل به صورت زیر است:

Books.csv
book_id , price
1,200
1,220
1,210
2,150
2,160

فایل دوم شامل امتیاز کاربران به کتاب‌ها است. فرض می‌شود که در هر فایل، داده‌ها بر اساس book_id مرتب شده‌اند و تمام رکوردهای مربوط به یک شناسه پشت سر هم قرار دارند. این ویژگی امکان پردازش جریانی داده‌ها بدون نیاز به نگهداری کامل فایل در حافظه را فراهم می‌کند. هر کتاب ممکن است چندین امتیاز از کاربران مختلف داشته باشد. فرمت هر سطر در این فایل به صورت زیر است:

Ratings.csv
book_id , rating
1,4.5
1,5
1,4
2,3.5
2,4
2,1

نحوه پیاده‌سازی

در ابتدا یک پردازه اصلی اجرا می‌شود که وظیفه‌ی مدیریت کل سیستم را بر عهده دارد. این پردازه با استفاده از fork و exec سه پردازه‌ی فرزند ایجاد می‌کند. ارتباط میان پردازه‌ها باید تنها از طریق pipe انجام شود. وظایف هر پردازنده به شرح زیر است:

۱- پردازه اصلی (Father Process) وظیفه مدیریت کلی اجرای برنامه را بر عهده دارد. این پردازه در ابتدا نام فایل‌های CSV را از ورودی برنامه دریافت می‌کند و سپس نام فایل Books.csv را برای پردازه Stage 1 و نام فایل Ratings.csv را برای پردازه Stage 3 ارسال می‌کند تا هر کدام عملیات مربوط به خود را انجام دهند. پس از پردازش داده‌ها در مراحل مختلف، نتایج نهایی توسط پردازه Stage 2 تولید شده و به پردازه اصلی بازگردانده می‌شود. در نهایت، پردازه اصلی این نتایج را دریافت کرده و آن‌ها را در خروجی ترمینال نمایش می‌دهد. علاوه بر این، این پردازه مسئول مدیریت شروع و پایان اجرای سایر پردازه‌ها نیز هست و هماهنگی لازم میان آن‌ها را برقرار می‌کند. پردازه Father صرفاً نقش هماهنگ‌کننده دارد و عملیات پردازش داده‌ها در Stage‌ها انجام می‌شود.

۲- پردازه Stage 1 وظیفه تحلیل قیمت کتاب‌ها را بر عهده دارد. این پردازه فایل Books.csv را خوانده و داده‌ها را به صورت جریانی پردازش می‌کند تا نیاز به بارگذاری کامل فایل در حافظه نباشد. در ابتدا فایل به صورت خط به خط خوانده می‌شود و رکوردهای مربوط به هر book_id جمع‌آوری می‌شوند. پس از جمع‌آوری رکوردهای مربوط به یک کتاب، میانگین قیمت آن محاسبه می‌شود. زمانی که خواندن رکوردهای مربوط به آن book_id به پایان رسید، نتیجه محاسبه شده بلافاصله به پردازه Stage 2 ارسال می‌شود تا در مراحل بعدی مورد استفاده قرار گیرد. برای مثال اگر داده‌های زیر خوانده شوند:

1,200

1,220

1,210

میانگین قیمت کتاب 1 محاسبه شده و بلافاصله به Stage 2 ارسال می‌شود.

پردازه Stage 3 وظیفه تحلیل امتیاز کاربران را بر عهده دارد و فایل Ratings.csv را به صورت جریانی پردازش می‌کند. در این پردازه، فایل خط به خط خوانده شده و تمام امتیازهای مربوط به هر book_id جمع‌آوری می‌شوند. پس از جمع‌آوری داده‌های امتیازی برای یک کتاب، میانگین امتیاز کاربران برای آن محاسبه شده و نتیجه حاصل برای آن کتاب خاص، جهت انجام تحلیل‌های تکمیلی به پردازه Stage 2 ارسال می‌شود.

پردازه Stage 2 نتایج حاصل از پردازه‌های Stage 1 و Stage 3 را دریافت و ترکیب می‌کند. این پردازه دارای دو صف داخلی است: یکی برای ذخیره نتایج میانگین قیمت کتاب‌ها که از Stage 1 دریافت می‌شود و دیگری برای نتایج میانگین امتیاز کاربران که از Stage 3 می‌رسد. هر بار که برای یک book_id معین، هر دو مقدار

میانگین قیمت و میانگین امتیاز کاربران موجود باشد، پردازش Stage 2 اقدام به محاسبه امتیاز نهایی آن کتاب می‌کند. این امتیاز نهایی براساس معیار زیر محاسبه و برای نمایش به پردازش اصلی ارسال شود:

$$\text{BookScore} = \text{MeanPrice} / \text{MeanRating}$$

برای مثال اگر داده‌های زیر دریافت شود:

از Stage 1:

book_id = 1

mean_price = 210

از Stage 3:

book_id = 1

mean_rating = 5

Stage 2 امتیاز نهایی کتاب که ۴۲ است را محاسبه می‌کند.

پس از محاسبه، نتیجه برای پردازش پدر ارسال می‌شود.

پردازش On-the-fly یکی از ویژگی‌های مهم این پروژه است که بر انجام پردازش به صورت همزمان و در حین خواندن داده‌ها تأکید دارد. در این روش، پردازش‌ها نباید کل فایل‌های مربوطه را در حافظه ذخیره کنند یا منتظر اتمام کامل فایل باشند. به محض اینکه داده‌های مربوط به یک book_id در هر یک از پردازش‌ها آماده شد، پردازش Stage 2 فوراً اقدام به محاسبه نتیجه نهایی می‌کند و نتیجه را به پردازش پدر ارسال می‌نماید. این رویکرد باعث کاهش زمان کلی پردازش و افزایش کارایی سیستم می‌شود، زیرا نیازی به انتظار برای کامل شدن تمام داده‌ها نیست و هر کتاب به سرعت و در همان لحظه تحلیل می‌شود که داده‌های آن به صورت همزمان در مدار قرار گرفته است.

نحوه ارتباط بین پردازش‌ها

در این پروژه ارتباط بین پردازش‌ها باید صرفاً از طریق Pipe انجام شود. برای پیاده‌سازی این ارتباط، از دو نوع Pipe استفاده خواهد شد:

- Unnamed Pipe

- Named Pipe

هر یک از این روش‌ها برای نوع خاصی از ارتباط بین پردازش‌ها مورد استفاده قرار می‌گیرند.

از Unnamed Pipe برای ارتباط بین پردازش پدر (Father) و پردازش‌های فرزند استفاده می‌شود. این نوع pipe برای ارتباط‌های مستقیم در ساختار پدر-فرزندی مناسب است.

در این پروژه، Unnamed Pipe در موارد زیر استفاده می‌شود:

- ارسال نام فایل‌ها از Father به پردازش‌های Stage 1 و Stage 3
- ارسال نتایج نهایی پردازش از Stage 2 به Father

از Named Pipe یا FIFO برای ارتباط بین پردازش‌هایی استفاده می‌شود که به صورت مستقیم در یک رابطه‌ی ساده‌ی پدر-فرزندی قرار ندارند و نیاز به یک کانال ارتباطی مستقل دارند.

در این پروژه، Named Pipe برای انتقال داده بین مراحل پردازش استفاده می‌شود:

- ارسال نتایج پردازش از Stage 1 به Stage 2
- ارسال نتایج پردازش از Stage 2 به Stage 3

ثبت Log

برای هر عملیات مهم در سیستم باید log مناسب چاپ شود.

برای مثال:

- ایجاد پردازش
- ارسال داده از یک پردازش به پردازش دیگر
- دریافت داده
- شروع و پایان پردازش
- پایان اجرای پردازش‌ها

وجود log مناسب برای بررسی صحت اجرای برنامه الزامی است.

برای ایجاد این نوع pipe می‌توان از فراخوانی سیستمی ``mkfifo`` استفاده کرد.

ساختار پوشه‌بندی پروژه

ساختار پروژه باید به صورت منظم و ماژولار طراحی شود. ساختار پیشنهادی پروژه به شکل زیر است:

Project Structure

```
ProjectRoot/  
|  
├── main.cpp  
├── Makefile  
|  
├── include/  
|   ├── stage1.cpp  
|   ├── stage1.hpp  
|   ├── stage2.cpp  
|   ├── stage2.hpp  
|   ├── stage3.cpp  
|   └── stage3.hpp
```

توضیحات:

- فایل `main.cpp` در ریشه پروژه قرار دارد و مسئول:
 - ایجاد پردازنده‌ها
 - ایجاد pipe ها
 - مدیریت اجرای کلی برنامه
 - هر Stage دارای یک فایل `cpp` و یک فایل `hpp` مجزا است.
 - پوشه `include` شامل پیاده‌سازی و تعریف توابع مربوط به هر پردازنده می‌باشد.
- این ساختار به منظور جداسازی منطقی مراحل مختلف pipeline در فایل‌های مستقل در نظر گرفته شده است.

سؤالات تحلیلی (پاسخ به صورت متنی الزامی است)

1. چرا برای ارسال نام فایل‌ها از پردازش Father به Stage 1 و Stage 3 از Unnamed Pipe استفاده شده است؟
2. چرا برای ارتباط بین Stage 1 و Stage 2 و همچنین Stage 2 و Stage 3 از Named Pipe استفاده شده است؟
3. اگر به جای Named Pipe از Unnamed Pipe برای ارتباط Stage 1 و Stage 2 استفاده می‌شد، چه محدودیت‌ها یا مشکلاتی به وجود می‌آمد؟
4. مزیت پردازش on-the-fly در این پروژه نسبت به پردازش پس از خواندن کامل فایل چیست؟
5. Stage 2 چگونه اطمینان حاصل می‌کند که برای یک book_id، داده‌های هر دو Stage 1 و Stage 3 دریافت شده‌اند؟

نکات پایانی و نحوه تحویل

- برنامه باید با استفاده از makefile و کامپایلر ++g کامپایل شود.
- برنامه باید در سیستم عامل لینوکس اجرا شود.
- ساختار پروژه باید شامل فایل‌های مربوط به هر پردازش باشد.
- تمامی فایل‌ها در قالب یک فایل فشرده تحویل داده شوند.
- برای ایجاد پردازش‌ها حتماً از fork و exec استفاده شود.
- به ازای هر پردازش در سیستم حداقل یک فایل cpp جداگانه در پروژه وجود داشته باشد.
- تنها روش مجاز برای ارتباط بین پردازش‌ها استفاده از pipe است.
- فرمت داده‌های ارسالی بین پردازش‌ها بر عهده دانشجو است.
- پس از اتمام استفاده از pipe ها باید آن‌ها را با close ببندید.
- در صورت استفاده از Named Pipe، پس از پایان برنامه باید فایل FIFO حذف شود.
- استفاده از thread مجاز نیست و هماهنگی باید صرفاً با process و pipe انجام شود.
- برای book_id هر تنها یک خروجی نهایی باید تولید شود.

